

SEAGAS

Student Employability Assessment & Gap Analysis System

Development Guide

Unit	COS40005 Computing Technology Project A
Group	Group 10
University	Swinburne University / INTI International College Subang
Branch	dev (GitHub: Group10_ProjectA)
Date	September 2026

This guide documents everything completed in Sprint 1 and provides step-by-step instructions for all remaining development tasks. All team members should read the setup section before starting development.

1. Team & Roles

Member	Role	Primary Responsibility
Shanice (Yan Min Xuan)	Team Leader + Report Lead	UI/UX wireframe, Student Dashboard, Report
Aaron (Tan Jun Xiong)	Co-Leader + Recommendation Engine	Rule-based engine, Dev tracking, Ethics
Carl (Soh Way Miin)	Backend Developer	FastAPI, PostgreSQL, Security, NLP integration
Ren Hang (Tee Ren Hang)	Frontend Developer + UI/UX	React, Advisor Dashboard, Chart.js
Yong Hin (Ng Yong Hin)	AI / NLP Engineer	spaCy, Sentence Transformers, Dataset

Tech Stack

Layer	Technology
Frontend	React.js, Chart.js, Plotly
Backend	Python, FastAPI
AI / NLP	spaCy, Sentence Transformers (all-MiniLM-L6-v2), TF-IDF, scikit-learn
Database	PostgreSQL 17
Dev Tools	GitHub (dev/main branches), Docker, VS Code
References	IEEE format

2. What We Completed (Sprint 1 — Sept 25, 2026)

2.1 Interactive System Prototype

Built a fully interactive HTML prototype of the SEAGAS system with all screens: Login (3 roles), Student Dashboard, Profile, Job Analysis, Skill Gap, Career Paths, Recommendations, Progress Tracking, and Advisor Dashboard. Published at [claude.ai](#).

2.2 PostgreSQL Database (14 Tables)

Table	Purpose	Key FR
users	All accounts (student/advisor/admin) + auth	NFR-04
student_profiles	Academic background, GPA, target role	FR-01
skills_library	100-150 IT skills, 4 categories, aliases	FR-02
student_skills	Student x Skill junction + proficiency	FR-01,02
job_roles	6 IT role templates (Data Analyst etc.)	FR-03
job_role_skills	Required skills per role	FR-03
job_descriptions	Raw JD text + NLP status	FR-03,04
jd_extracted_skills	NLP-extracted skills from JD	FR-04
assessments	Overall score + 7-dimension breakdown	FR-07,08
skill_match_results	Per-skill strong/developing/gap result	FR-09
recommendations_library	Curated courses, certs, projects	FR-10
student_recommendations	Student tracking + completion status	FR-10,11
student_projects	Student project records	FR-01
student_certifications	Student certification records	FR-01

Seed data included: 6 IT job roles + 32 skills pre-loaded. 3 Views created: `v_student_assessment_summary`, `v_cohort_skill_gaps`, `v_student_progress`.

2.3 FastAPI Backend (5 Routers, 25+ Endpoints)

Router	File	Key Endpoints
Authentication	<code>routers/auth.py</code>	POST <code>/register</code> , POST <code>/login</code> , GET <code>/me</code>
Students	<code>routers/students.py</code>	Profile CRUD, Skills, Projects, Certifications

Jobs	routers/jobs.py	JD Submit, NLP Trigger, Role Templates
Assessments	routers/assessments.py	Run Assessment, Get Results, Recommendations
Advisor	routers/advisor.py	Cohort Overview, Skill Gaps, Student List

API running at <http://127.0.0.1:8000> — Swagger UI at <http://127.0.0.1:8000/docs>

3. Environment Setup (Every Team Member)

IMPORTANT: Every team member must complete ALL steps below before writing any code. The .env file must be created manually — it is NOT in GitHub (for security).

Step 1 — Clone the Repository

```
git clone https://github.com/j23039278-netizen/Group10_ProjectA.git

cd Group10_ProjectA

git checkout dev
```

Step 2 — Install PostgreSQL 17

Download from: <https://www.postgresql.org/download/windows/>

Select version 17.x, Windows x86-64. During install:

- Set a password for the postgres superuser (remember this!)
- Port: 5432 (default, do not change)
- Add to PATH: C:\Program Files\PostgreSQL\17\bin

Verify installation:

```
psql --version

# Expected: psql (PostgreSQL) 17.x
```

Step 3 — Create Database & Run Schema

```
psql -U postgres

# Enter your password

CREATE DATABASE seagas_db;

\q

# Run schema (from project root folder):

psql -U postgres -d seagas_db -f database/schema.sql

# Verify (should show 14 tables):

psql -U postgres -d seagas_db -c "\dt"
```

Step 4 — Install Python Dependencies

```
pip install fastapi uvicorn sqlalchemy "psycopg[binary]" python-dotenv
```

```
alembic bcrypt "python-jose[cryptography]" "pydantic[email]" faker
```

Step 5 — Create .env File

Create a file named .env inside the backend/ folder:

```
# backend/.env

DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@localhost:5432/seagas_db

SECRET_KEY=seagas_super_secret_key_change_in_production_2026

ALGORITHM=HS256

ACCESS_TOKEN_EXPIRE_MINUTES=30
```

Replace YOUR_PASSWORD with the password you set during PostgreSQL installation. NEVER commit .env to GitHub — it is already in .gitignore.

Step 6 — Test Backend

```
cd backend

uvicorn main:app --reload

# Expected output:

# INFO: Uvicorn running on http://127.0.0.1:8000

# INFO: Application startup complete.

# Open browser: http://127.0.0.1:8000/docs
```

4. What to Do Next — Sprint 2 (Week 9-10)

Step	Task	Who	File Location
6	NLP Module — JD skill extraction	Yong Hin	nlp/extractor.py
7	Backend API full implementation	Carl	backend/routers/
8	Rule-based Recommendation Engine	Aaron	recommendation/engine.py
9	React Frontend	Ren Hang + Shanice	frontend/src/

Step 6 — NLP Module (Yong Hin)

Install NLP dependencies:

```
pip install spacy sentence-transformers scikit-learn

python -m spacy download en_core_web_sm
```

Create nlp/extractor.py:

```
"""

SEAGAS NLP Module — JD Skill Extraction

Author: Ng Yong Hin (AI/NLP Engineer)

"""

import spacy

from sentence_transformers import SentenceTransformer

from sklearn.metrics.pairwise import cosine_similarity

import numpy as np

nlp = spacy.load('en_core_web_sm')

model = SentenceTransformer('all-MiniLM-L6-v2')

SKILL_KEYWORDS = [

    'python', 'sql', 'javascript', 'react', 'power bi', 'tableau',

    'machine learning', 'deep learning', 'cloud', 'aws', 'azure',

    'docker', 'git', 'linux', 'data visualization', 'nlp',

    'communication', 'problem solving', 'teamwork',
```

```

]

def extract_skills_level1(jd_text: str) -> list:

    '''Level 1: Keyword matching'''

    text_lower = jd_text.lower()

    found = []

    for skill in SKILL_KEYWORDS:

        if skill in text_lower:

            found.append({'skill': skill, 'method': 'keyword', 'confidence': 1.0})

    return found

def extract_skills_level2(jd_text: str, threshold=0.65) -> list:

    '''Level 2: Semantic similarity using Sentence Transformers'''

    doc = nlp(jd_text)

    noun_chunks = [chunk.text for chunk in doc.noun_chunks]

    if not noun_chunks:

        return extract_skills_level1(jd_text)

    chunk_embeddings = model.encode(noun_chunks)

    skill_embeddings = model.encode(SKILL_KEYWORDS)

    similarities = cosine_similarity(chunk_embeddings, skill_embeddings)

    found = []

    for i, chunk in enumerate(noun_chunks):

        best_idx = np.argmax(similarities[i])

        best_score = similarities[i][best_idx]

        if best_score >= threshold:

            found.append({

                'raw_text': chunk,

                'skill': SKILL_KEYWORDS[best_idx],

                'method': 'semantic',

                'confidence': round(float(best_score), 3),

```



```

}))

return found

def extract_skills(jd_text: str, method='semantic') -> list:

    if method == 'keyword':

        return extract_skills_level1(jd_text)

    return extract_skills_level2(jd_text)

```

Create nlp/matcher.py:

```

"""

SEAGAS Skill Matcher – compares student skills vs JD requirements

Author: Ng Yong Hin

"""

from sentence_transformers import SentenceTransformer

from sklearn.metrics.pairwise import cosine_similarity

import numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')

def match_skills(student_skills: list, required_skills: list,

method='semantic', threshold=0.65) -> list:

    """

    Compare student skills against required skills.

    Returns list of match results with status: strong/developing/gap

    """

    results = []

    if method == 'keyword':

        student_lower = [s.lower() for s in student_skills]

        for req in required_skills:

            if req.lower() in student_lower:

                status, score = 'strong', 1.0

```

```

else:

    status, score = 'gap', 0.0

    results.append({'skill': req, 'status': status, 'score': score})

else:

    req_emb = model.encode(required_skills)

    std_emb = model.encode(student_skills)

    sim_matrix = cosine_similarity(req_emb, std_emb)

    for i, req in enumerate(required_skills):

        best_score = float(np.max(sim_matrix[i]))

        if best_score >= 0.85:

            status = 'strong'

        elif best_score >= threshold:

            status = 'developing'

        else:

            status = 'gap'

        results.append({'skill': req, 'status': status, 'score': round(best_score,3)})

    return results


def calculate_readiness_score(match_results: list) -> dict:

    '''Calculate Job Readiness Score from match results'''

    total = len(match_results)

    strong = sum(1 for r in match_results if r['status'] == 'strong')

    developing= sum(1 for r in match_results if r['status'] == 'developing')

    gap = sum(1 for r in match_results if r['status'] == 'gap')

    score = (strong * 1.0 + developing * 0.5) / total * 100 if total > 0 else 0

    return {

        'readiness_score': round(score, 2),

        'strong': strong, 'developing': developing, 'gap': gap, 'total': total

    }

```

Step 7 — Backend: Connect NLP to API (Carl)

In backend/routers/jobs.py, replace the TODO comment in trigger_nlp_analysis() with:

```
# In trigger_nlp_analysis() – replace TODO block:

import sys, os

sys.path.append(os.path.join(os.path.dirname(__file__), '..', '..'))

from nlp.extractor import extract_skills

from nlp.matcher import match_skills, calculate_readiness_score


# Extract skills from JD

extracted = extract_skills(jd.raw_text, method='semantic')


# Save extracted skills to DB

for item in extracted:

    # Lookup skill_id from skills_library

    skill_row = db.execute(

        text('SELECT skill_id FROM skills_library WHERE LOWER(skill_name)=:name'),

        {'name': item['skill'].lower()})

    .fetchone()

    db.execute(

        text('''INSERT INTO jd_extracted_skills

(jd_id, skill_id, raw_skill_text, extraction_method, confidence_score)

VALUES (:jid, :sid, :raw, :method, :conf)'''),

        {'jid': jd_id, 'sid': str(skill_row.skill_id) if skill_row else None,

'raw': item.get('raw_text', item['skill']),

'method': item['method'], 'conf': item['confidence']})

    )

db.commit()
```

Step 8 — Recommendation Engine (Aaron)

Create recommendation/engine.py:

```
"""
```

```
SEAGAS Recommendation Engine – Rule-based
```

```
Author: Tan Jun Xiong (Aaron)
```

```
"""
```

```
from sqlalchemy.orm import Session
```

```
from sqlalchemy import text
```

```
def get_recommendations_for_gaps(db: Session, assessment_id: str) -> list:
```

```
    '''
```

```
    Rule: IF skill_status IN (gap, developing)
```

```
    THEN fetch resources from recommendations_library
```

```
    '''
```

```
    gaps = db.execute(text('''
```

```
    SELECT smr.skill_id, sl.skill_name, smr.match_status
```

```
    FROM skill_match_results smr
```

```
    JOIN skills_library sl ON sl.skill_id = smr.skill_id
```

```
    WHERE smr.assessment_id = :aid
```

```
    AND smr.match_status IN ('gap','developing')
```

```
    ORDER BY smr.match_status
```

```
    '''), {'aid': assessment_id}).fetchall()
```

```
    recommendations = []
```

```
    for gap in gaps:
```

```
        resources = db.execute(text('''
```

```
        SELECT * FROM recommendations_library
```

```
        WHERE skill_id = :sid AND is_active = TRUE
```

```
        ORDER BY difficulty LIMIT 3
```

```
        '''), {'sid': str(gap.skill_id)}).fetchall()
```

```
        recommendations.append({
```

```
            'skill_name': gap.skill_name,
```

```

'gap_type': gap.match_status,

'resources': [dict(r._mapping) for r in resources]

})

return recommendations

def save_recommendations(db: Session, profile_id: str,
assessment_id: str, recommendations: list):

'''Save generated recommendations to student_recommendations table'''

for rec in recommendations:

for resource in rec['resources']:

db.execute(text('''

INSERT INTO student_recommendations

(assessment_id, profile_id, resource_id, status)

VALUES (:aid, :pid, :rid, 'not_started')

ON CONFLICT (assessment_id, resource_id) DO NOTHING

'''), {

'aid': assessment_id,

'pid': profile_id,

'rid': str(resource['resource_id'])

})

db.commit()

```

Populate the recommendations_library with resources. Example SQL:

```

-- Add resources for Power BI gap

INSERT INTO recommendations_library

(skill_id, resource_title, resource_type, provider, url, duration_hours, cost, difficulty)

SELECT sl.skill_id,

'Microsoft Power BI Fundamentals',

'course', 'Microsoft Learn',

'https://learn.microsoft.com/en-us/training/powerbi/',

```

```
8, 'Free', 'beginner'

FROM skills_library sl WHERE sl.skill_name = 'Power BI';

-- Add resources for Cloud gap

INSERT INTO recommendations_library

(skill_id, resource_title, resource_type, provider, url, duration_hours, cost, difficulty)

SELECT sl.skill_id,

'AWS Cloud Practitioner Certification',

'certification', 'AWS Skill Builder',

'https://aws.amazon.com/certification/certified-cloud-practitioner/',

40, '$100 USD', 'beginner'

FROM skills_library sl WHERE sl.skill_name = 'Cloud Computing';
```

Step 9 — React Frontend (Ren Hang + Shanice)

Install React:

```
# From project root

cd frontend

npx create-react-app . --template typescript

npm install axios react-router-dom recharts

npm install -D tailwindcss postcss autoprefixer

npx tailwindcss init -p
```

Create frontend/src/api/client.js (API connection):

```
// frontend/src/api/client.js

import axios from 'axios';

const API = axios.create({

  baseURL: 'http://127.0.0.1:8000/api',

});

// Auto-attach JWT token to every request

API.interceptors.request.use(config => {

  const token = localStorage.getItem('token');

  if (token) config.headers.Authorization = `Bearer ${token}`;

  return config;

});

export const authAPI = {

  login: (email, password) =>

    API.post('/auth/login', new URLSearchParams({username:email, password})),

  register: (data) => API.post('/auth/register', data),

  me: () => API.get('/auth/me'),

};

export const studentAPI = {
```

```

getProfile: () => API.get('/students/profile'),

createProfile: (data) => API.post('/students/profile', data),

updateProfile: (data) => API.put('/students/profile', data),

getSkills: () => API.get('/students/skills'),

addSkill: (data) => API.post('/students/skills', data),

getSkillLibrary: () => API.get('/students/skills-library'),

};

export const jobsAPI = {

getRoles: () => API.get('/jobs/roles'),

submitJD: (data) => API.post('/jobs/descriptions', data),

analyseJD: (jdId) => API.post(`/jobs/descriptions/${jdId}/analyse`),

};

export const assessAPI = {

run: (data) => API.post('/assessments/run', data),

get: (id) => API.get(`/assessments/${id}`),

history: () => API.get('/assessments/my/history'),

recs: (id) => API.get(`/assessments/${id}/recommendations`),

};

export const advisorAPI = {

overview: () => API.get('/advisor/cohort/overview'),

skillGaps: () => API.get('/advisor/cohort/skill-gaps'),

students: () => API.get('/advisor/students'),

};

```

Create frontend/src/pages/Login.jsx:

```

import { useState } from 'react';

import { authAPI } from '../api/client';

export default function Login({ onLogin }) {

```



```

const [email, setEmail] = useState('');

const [password, setPassword] = useState('');

const [error, setError] = useState('');

const handleSubmit = async (e) => {

  e.preventDefault();

  try {

    const res = await authAPI.login(email, password);

    const data = res.data;

    localStorage.setItem('token', data.access_token);

    localStorage.setItem('role', data.role);

    onLogin(data.role);

  } catch (err) {

    setError('Invalid email or password');

  }

};

return (

  <div className='min-h-screen flex items-center justify-center bg-gray-50'>

    <form onSubmit={handleSubmit} className='bg-white p-8 rounded-xl shadow w-96'>

      <h1 className='text-2xl font-bold mb-6'>SEAGAS Login</h1>

      {error && <p className='text-red-500 mb-4'>{error}</p>}}

      <input type='email' placeholder='Email'

        className='w-full border p-2 rounded mb-3'

        value={email} onChange={e => setEmail(e.target.value)} />

      <input type='password' placeholder='Password'

        className='w-full border p-2 rounded mb-4'

        value={password} onChange={e => setPassword(e.target.value)} />

      <button type='submit'

        className='w-full bg-blue-600 text-white py-2 rounded hover:bg-blue-700'>

```

Sign In

</button>

</form>

</div>

);

}

5. Generate Synthetic Dataset (Yong Hin)

Install Faker then run the generator:

```
pip install faker

# Put generate_synthetic_data.py in the data/ folder

cd data

python generate_synthetic_data.py

# Output files:

# students.csv – 3000 student profiles

# student_skills.csv – skill entries per student

# summary_stats.txt – distribution statistics
```

To generate 5000 records instead of 3000, change NUM_STUDENTS = 5000 on line 23 of generate_synthetic_data.py

6. Full Sprint Plan

Sprint	Weeks	Task	Who	Status
1	6-8	Database Schema (14 tables)	Carl	DONE
1	6-8	FastAPI Backend Skeleton	Carl	DONE
1	6-8	Interactive Prototype	All	DONE
1	6-8	UI/UX Wireframes	Shanice	In Progress
2	9-10	NLP Module (spaCy + Sentence Transformers)	Yong Hin	TODO
2	9-10	Synthetic Dataset Generation	Yong Hin	TODO
2	9-10	Skill Library (100-150 skills)	Yong Hin + Aaron	TODO
2	9-10	Backend API Full Implementation	Carl	TODO
2	9-10	Recommendation Engine + Resource Library	Aaron	TODO
2	9-10	React Frontend (all pages)	Ren Hang + Shanice	TODO
3	11-12	System Integration (frontend + backend)	All	TODO
3	11-12	End-to-end Testing	All	TODO

3	11-12	Bug Fixes + Edge Cases	Carl	TODO
4	13-14	AI Evaluation (L1 vs L2 comparison)	Yong Hin	TODO
4	13-14	Usability Evaluation (SUS questionnaire)	All	TODO
4	13-14	Final Report + Demo	Shanice (lead)	TODO

7. Git Workflow

Always work on dev branch. Never push directly to main. main is only updated when a Sprint is complete and tested.

```
# Before starting work each day:

git checkout dev

git pull origin dev


# Create a feature branch for your task:

git checkout -b feature/nlp-extractor # Yong Hin
git checkout -b feature/recommendation-engine # Aaron
git checkout -b feature/student-dashboard # Shanice
git checkout -b feature/advisor-dashboard # Ren Hang


# After finishing your task:

git add .

git commit -m "feat: [your task description]"

git push origin feature/your-branch-name


# Then create a Pull Request to merge into dev on GitHub
```